



UNIVERSITY OF LEEDS

School of Mathematics

Declaration of Academic Integrity for Individual Pieces of Work

I am aware that the University defines plagiarism as presenting someone else's work as your own. Work means any intellectual output, and typically includes text, data, images, sound or performance.

I promise that in the attached submission I have not presented anyone else's work as my own and I have not colluded with others in the preparation of this work. Where I have taken advantage of the work of others, I have given full acknowledgement. I have read and understood the University's published rules on plagiarism and also any more detailed rules specified at School or module level. I know that if I commit plagiarism I can be expelled from the University and that it is my responsibility to be aware of the University's regulations on plagiarism and their importance.

I re-confirm my consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to monitor breaches of regulations, to verify whether my work contains plagiarised material, and for quality assurance purposes.

I confirm that I have declared all mitigating circumstances that may be relevant to the assessment of this piece of work and that I wish to have taken into account. I am aware of the School's policy on mitigation and procedures for the submission of statements and evidence of mitigation. I am aware of the penalties imposed for the late submission of coursework.

Student Signature  Date 07/05/2025

Student Name Jack Adams Student Number 201885202

Please note.

When you become a registered student of the University at first and any subsequent registration you sign the following authorisation and declaration:

"I confirm that the information I have given on this form is correct. I agree to observe the provisions of the University's Charter, Statutes, Ordinances, Regulations and Codes of Practice for the time being in force. I know that it is my responsibility to be aware of their contents and that I can read them on the University web site. I acknowledge my obligation under the Payment of Fees Section in the Handbook to pay all charges to the University on demand.

I agree to the University processing my personal data (including sensitive data) in accordance with its Code of Practice on Data Protection <http://www.leeds.ac.uk/dpa>. I consent to the University making available to third parties (who may be based outside the European Economic Area) any of my work in any form for standards and monitoring purposes including verifying the absence of plagiarised material. I agree that third parties may retain copies of my work for these purposes on the understanding that the third party will not disclose my identity."

Introduction

In this report, we solve the Black-Scholes PDE for a bounded power put option using the implicit method. We also use interpolation to estimate the option price and compute the Greek Θ , i.e. the partial derivative of the option value with respect to time. Before implementing the implicit method, we construct a function to solve a tridiagonal system of linear equations and derive the appropriate boundary conditions for the option, as both are essential components of the numerical scheme.

Linear System Solver

In this section, we build a function `tridiagonal_solve(lambda_, b)` to find the solution to the following linear system of equations:

$$\begin{bmatrix} 1+2\lambda & -\lambda & 0 & \cdots & 0 \\ -\lambda & 1+2\lambda & -\lambda & \ddots & \vdots \\ 0 & -\lambda & 1+2\lambda & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -\lambda \\ 0 & \cdots & 0 & -\lambda & 1+2\lambda \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix} \quad (1)$$

Rather than storing the entire matrix, which would mostly consist of zeroes, we store the diagonals in a banded format. This format is compatible with the `solve_banded` function from the `linalg` module of `scipy`, and is both memory efficient and computationally faster than handling the full matrix directly. The `solve_banded` function can then be used to efficiently compute the solution to linear systems of the required form. This function, summarised in **Algorithm 1**, is essential for later sections, where it will be used in the implicit method for solving the discretised heat equation.

Algorithm 1 Solve tridiagonal system (1) using banded matrix form

Input: λ , right-hand side vector $\mathbf{b} \in \mathbb{R}^n$

Set upper diagonal: `upper` = $-\lambda \cdot \mathbf{1}_{n-1}$
Set main diagonal: `main` = $(1 + 2\lambda) \cdot \mathbf{1}_n$
Set lower diagonal: `lower` = $-\lambda \cdot \mathbf{1}_{n-1}$
Construct banded matrix:

$$\mathbf{ab} = \begin{bmatrix} 0 & \text{upper}_1 & \cdots & \text{upper}_{n-1} \\ \text{main}_0 & \cdots & \cdots & \text{main}_{n-1} \\ \text{lower}_0 & \cdots & \text{lower}_{n-2} & 0 \end{bmatrix}$$

Solve $\mathbf{ab} \cdot \mathbf{x} = \mathbf{b}$ using `solve_banded((1,1), ab, b)`

Output: Solution vector $\mathbf{x} \in \mathbb{R}^n$

To ensure the correctness of `tridiagonal_solve(lambda_, b)`, we validate it using a simple linear system with a known solution. We choose $\lambda = 1$ and consider a system of size $n = 4$. The corresponding tridiagonal matrix A is:

$$A = \begin{bmatrix} 3 & -1 & 0 & 0 \\ -1 & 3 & -1 & 0 \\ 0 & -1 & 3 & -1 \\ 0 & 0 & -1 & 3 \end{bmatrix} \quad (2)$$

We define the true solution vector as:

$$\mathbf{x}_{\text{true}} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \quad (3)$$

We compute the vector $\mathbf{b}$ by multiplying A with the known solution $\mathbf{x}_{\text{true}}$. Then, we apply our solver function to compute the numerical solution. Comparing the result with the known solution, we find they are exactly the same, thereby validating the correctness of `tridiagonal_solve(lambda_, b)`.

Bounded Power Put Option Boundary Conditions

In this section, we consider the boundary conditions of a bounded power put option, where the pay-off at the maturity time T given a stock price S is:

$$h(S) = \min(L, ((K - S)^+)^p) \quad (4)$$

where $L > 0$ is the upper bound, $K > 0$ is the strike price, $p > 0$ is the power. We assume $L < K^p$ when $p \neq 1$.

Black-Scholes PDE

We consider the standard Black-Scholes market model where there is one bond, priced at $B_t = e^{rt}$, and one stock, whose price process under the risk neutral measure satisfies:

$$dS_t = rS_t dt + \sigma S_t dW_t \quad (5)$$

where W_t is a Brownian motion, r is the interest rate and σ is a volatility.

The price of an option on this stock satisfies the following partial differential equation:

$$\frac{\partial V(S, t)}{\partial t} + rS \frac{\partial V(S, t)}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rV(S, t) = 0, \quad (6)$$

where $V(S, t)$ is the price of an option as a function of the stock price and time.

We first look at the boundary conditions accompanying the Black-Scholes PDE. The terminal condition is simply the payoff at maturity:

$$V(S, T) = h(S) = \min(L, ((K - S)^+)^p) \quad (7)$$

As S approaches zero, the option is deep in-the-money, so we have the following left boundary condition:

$$\lim_{S \rightarrow 0} V(S, t) = e^{-r(T-t)} \min(L, K^p) \quad (8)$$

As S approaches infinity, the option is deep out-of-the-money, so we have the following right boundary condition:

$$\lim_{S \rightarrow \infty} V(S, t) = 0 \quad (9)$$

These conditions form the complete specification for solving the Black-Scholes PDE prior to transformation.

The Heat Equation

We now transform the Black-Scholes PDE into the heat equation using the following change of variables:

$$\tau = \frac{1}{2} \sigma^2 (T - t), \quad x = \log S + \left(r - \frac{1}{2} \sigma^2\right) (T - t) \quad (10)$$

We define the transformed function $y(x, \tau)$ by:

$$y(x, \tau) = \exp\left(-r\left(T - \frac{2\tau}{\sigma^2}\right)\right) V\left(\exp\left(x - \left(\frac{2r}{\sigma^2} - 1\right)\tau\right), T - \frac{2\tau}{\sigma^2}\right) \quad (11)$$

Under this transformation, the function $y(x, \tau)$ satisfies the standard heat equation:

$$\frac{\partial y}{\partial \tau} - \frac{\partial^2 y}{\partial x^2} = 0 \quad (12)$$

This transformation is useful, as the heat equation is much easier to solve, both analytically and numerically.

At maturity $\tau = 0$, using the terminal condition from Equation (7) we obtain an initial condition as follows:

$$y(x, 0) = e^{-rT} \min(L, ((K - e^x)^+)^p), \quad \text{for all } x \in \mathbb{R} \quad (13)$$

As S approaches zero, x approaches negative infinity, giving us the following left boundary condition:

$$\lim_{x \rightarrow -\infty} y(x, \tau) = e^{-r(T - \frac{2\tau}{\sigma^2})} \min(L, K^p), \quad \text{for all } \tau \in \left[0, \frac{1}{2} \sigma^2 T\right] \quad (14)$$

As S approaches infinity, x also approaches infinity, giving us the following right boundary condition:

$$\lim_{x \rightarrow \infty} y(x, \tau) = 0, \quad \text{for all } \tau \in \left[0, \frac{1}{2} \sigma^2 T\right] \quad (15)$$

These conditions form the complete specification for solving the heat equation.

Numerical Solution of the Black-Scholes PDE - Implicit Method

In this section, we build and implement a function `bounded_power_implicit(r,sigma,K,L,p,T,M,N,xmin,xmax)` that solves the heat equation using the implicit method with boundary conditions corresponding to the bounded power put option. Our function returns the option price at the $(M+1) \times (N+1)$ grid points, as well as the corresponding stock prices and the time vector t . **Algorithm 2** summarises our approach.

Algorithm 2 Compute the price of a European bounded power put option using the implicit method

Input: $r, \sigma, K, L, p, T, M, N, x_{\min}, x_{\max}$

Set $T = T/12$, $\delta\tau = \frac{\sigma^2 T}{2N}$, $\delta x = \frac{x_{\max} - x_{\min}}{M}$, $\lambda = \frac{\delta\tau}{(\delta x)^2}$

Define grid points $x_i = x_{\min} + i\delta x$, $S_{i,\nu} = \exp\left(x_i - \left(\frac{2r}{\sigma^2} - 1\right)\tau_\nu\right)$ for $i = 0, \dots, M$

Define time grid $\tau_\nu = \nu\delta\tau$, $t_\nu = T - \tau_\nu$ for $\nu = 0, \dots, N$

Set $w_{i,0} = e^{-rT} \cdot \min(L, \max(K - S_{i,0}, 0)^p)$ for $i = 0$ to M

for $\nu = 0$ **to** $N - 1$ **do**

 Set $w_{0,\nu+1} = e^{-r(T-\tau_{\nu+1})} \min(L, K^p)$ (bounded power put boundary)

 Set $w_{M,\nu+1} = 0$

 Construct vector $b_j = w_{j,\nu}$ for $j = 1, \dots, M - 1$

 Update $b_1 = b_1 + \lambda \cdot w_{0,\nu+1}$

 Update $b_{M-1} = b_{M-1} + \lambda \cdot w_{M,\nu+1}$

 Solve for interior points: $w_{j,\nu+1} = \text{tridiagonal.solve}(\lambda, b)$ for $j = 1$ to $M - 1$

 Set $w_{i,\nu+1}$ for all $i = 0$ to M

end for

Output: $w_{i,\nu}$ (option values), $S_{i,\nu}$ (stock prices), $t_\nu = T - \tau_\nu$ (time grid)

Note that in our algorithm we use the approximation $T - \tau$ to define the time-dependent left boundary condition instead of the theoretically correct form $T - \frac{2}{\sigma^2}\tau$. This simplification avoids excessive flattening of the option value near $S = 0$ and improves numerical behaviour at the boundary, while still producing accurate results in the interior.

Using **Algorithm 2**, we compute the value of a vanilla European put option. The known parameters of the option are interest rate $r = 8\%$, volatility $\sigma = 10\%$, strike price $K = \text{€}10$, and maturity $T = 18$ months.

To use the bounded power put option framework to replicate a vanilla European put, we select suitable values of L and p . Setting $p = 1$ and $L > K$ ensures that the payoff function matches that of a vanilla European put. Therefore, we choose $L = \text{€}11$.

We next choose the number of spatial steps M (in the log-stock price axis x) and time steps N . A larger M improves the resolution of the option payoff, while a larger N enhances time discretisation. Although the implicit method is unconditionally stable, increasing N can still improve numerical accuracy. After testing various values, we proceed with $M = N = 100$, as higher values do not significantly improve accuracy but do increase computational cost.

Finally, we must define the bounds of the spatial domain, $x_{\min}$ and $x_{\max}$. These must be wide enough to capture reasonable movements in the underlying asset. We select:

$$x_{\min} = \ln(0.001 \cdot K), \quad x_{\max} = \ln(10 \cdot K) \quad (16)$$

This range ensures sufficient coverage of the option payoff domain without incurring excessive computational costs.

Using our selected parameters as input to **Algorithm 2**, we use the output to plot the option value at the current time $t = 0$ as a function of the stock price S , ranging from $\text{€}0$ to $\text{€}20$. To verify the correctness of our implementation, we compare the numerical solution with the analytical Black-Scholes formula for a European put option:

$$\text{price of put} = Ke^{-rT}\Phi(-d_2) - S_0\Phi(-d_1) \quad (17)$$

where

$$d_1 = \frac{\log(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}} \text{ and } d_2 = d_1 - \sigma\sqrt{T} \quad (18)$$

From Figure 1, we observe that our numerical solution very closely matches the true option value given by the Black-Scholes formula. This agreement confirms the accuracy of our implementation of the implicit method for solving the heat equation.

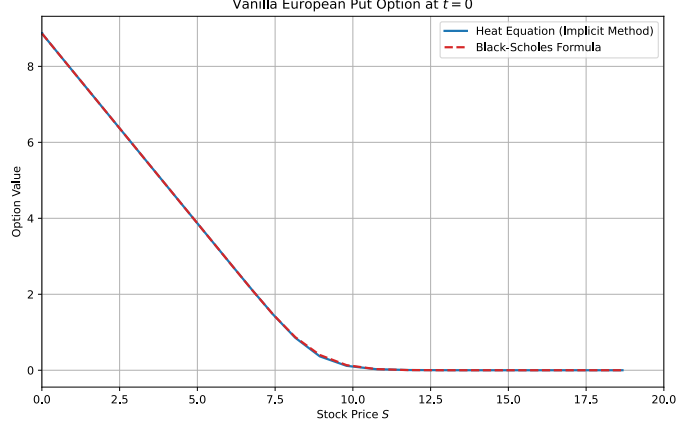


Figure 1: Comparison of the numerical solution of the heat equation using the implicit method with the analytical Black-Scholes formula for a European put option at time $t = 0$.

Analysis of the Case $p = 2$

In this section, we change the bounded power put option's upper bound to $L = \text{€}81$ and the power to $p = 2$. The other parameters remain the same as in the previous section. We begin by plotting a surface plot of the option value $V(S, t)$ as a function of stock price S and time t , computed using **Algorithm 2**, as shown in Figure 2.

The power parameter $p = 2$ gives the option a quadratic payoff structure, capped at $L = \text{€}81$. As expected, the option value is highest for low stock prices and decreases rapidly as S increases, with a significant drop occurring once the stock price exceeds the strike $K = \text{€}10$. We observe that the option value remains close to zero throughout the domain for stock prices near or above the strike, as the probability of the asset falling deep in-the-money is low and the payoff is zero in this region. A noticeable increase in option value only occurs in the deep in-the-money region (very low S) and becomes more pronounced as maturity approaches, due to the reduced discounting of potential payoff. This behaviour is consistent with the structure of a bounded power put option, reflecting the non-linear payoff growth and the effect of the upper cap on the option value.

Surface Plot of Bounded Power Put Option when $p = 2$ and $L = 81$

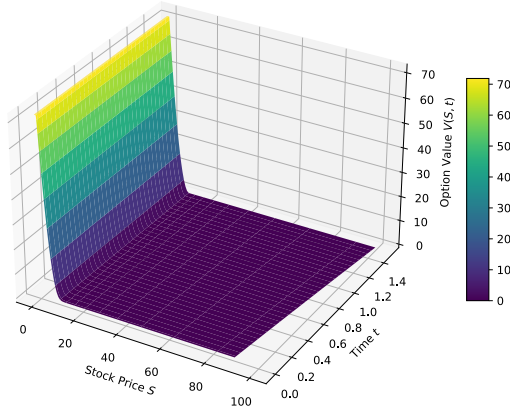


Figure 2: Surface plot of the bounded power put option value computed using the implicit method applied to the heat equation

We next compare the current value of the option, i.e. at $t = 0$, to the discounted payoff function, as shown in Figure 3. The numerical solution and the discounted payoff align almost perfectly, confirming that the implementation of the implicit method correctly computes the option value at maturity. This agreement is expected, since at $t = 0$, the value of a European-style option must satisfy the terminal condition:

$$V(S, 0) = e^{-rT} \cdot \min(L, \max(K - S, 0)^p), \quad (19)$$

which serves as the initial condition for the backward time-stepping scheme applied to the heat equation (see Equation (13)).

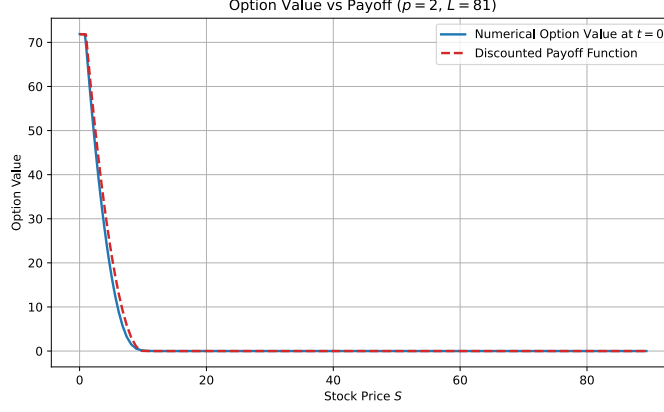


Figure 3: Comparison of the bounded power put option value and its payoff at $t = 0$

We go on to estimate the value of the option given a specific stock price of $S = \text{€}4.73$. Since the spatial grid used in the numerical solution may not contain this exact value of S , we use the `interp` function from the `numpy` package to perform linear interpolation. This allows us to estimate the option value at $t = 0$ based on the known grid of stock prices and the corresponding final-time option values computed via the implicit finite difference method applied to the heat equation.

Table 1, shows the results of our implementation. This value is expected, as the option is deep in-the-money with S significantly below the strike price $K = \text{€}10$. The power parameter $p = 2$ further amplifies the payoff when $K - S$ is large, due to the quadratic structure. The bounded nature of the payoff (capped at $L = \text{€}81$) has not yet been reached, so the payoff behaves like an unconstrained power put in this region. This validates both the magnitude of the estimated value and the interpolation method used.

S at $t = 0$ (€)	$V(S, 0)$ (€)
4.73	19.7194

Table 1: Estimated value of the bounded power put option with current stock price = €4.73

We now estimate the Greek Θ , which represents the sensitivity of the option value to the passage of time, we use a finite difference approximation based on the values obtained from the numerical solution to the Black–Scholes PDE via the implicit method. We approximate the partial derivative with respect to time using a forward differencing scheme:

$$\Theta(S_i) = \frac{\partial V}{\partial t}(S_i, t_0) \approx \frac{V(S_i, t_1) - V(S_i, t_0)}{\Delta t} \quad (20)$$

where t_0 is the final time (i.e., current time $t = 0$), t_1 is the previous time step, $\Delta t = t_1 - t_0$ is the time step size, $V(S_i, t_j)$ is the option value at stock price S_i and time t_j .

This method uses two adjacent time levels from the numerical solution matrix: the final column (corresponding to $t = 0$) and the penultimate column. The estimate $\Theta(S_i)$ is computed for each grid point S_i on the stock price axis.

The finite difference approximation for Θ is implemented using forward differencing between the final two time levels of the numerical grid. However, due to the transformation used in the heat-equation formulation, the stock price grid S evolves with time: $S_{i,\nu} = \exp(x_i - (\frac{2r}{\sigma^2} - 1)\tau_\nu)$. As a result, the stock prices at t_0 and t_1 differ, meaning the option values $V(S_i, t_0)$ and $V(S_i, t_1)$ are not naturally aligned. We therefore interpolate the option values from t_1 onto the S grid at t_0 before applying the finite difference approximation. This interpolation ensures a valid and accurate estimate of $\Theta(S)$ and is implemented using the `numpy.interp` function.

To validate our numerical approximation of Θ , we apply it to a European put option, for which an analytical expression is known under the Black–Scholes model. The analytical formula is given by:

$$\Theta = rKe^{-rT}\Phi(-d_2) - \frac{\sigma S\phi(d_1)}{2\sqrt{T}}, \quad (21)$$

where d_1 and d_2 are as seen in Equation (18).

As shown in Figure 4, both the numerical and analytical curves follow a similar shape, with the peak of Θ occurring at approximately the same stock price. However, the analytical curve exhibits a slightly higher maximum value. This difference can be attributed to the nature of the numerical method: our implementation uses finite differences to approximate time derivatives, which introduces a degree of smoothing, particularly around regions where Θ changes rapidly (i.e. near the strike price). Discretisation in both time and asset price, as well as interpolation between grids

at different time steps, may also contribute to minor deviations before the peak. After the peak, both curves align very closely, confirming that the numerical method captures the overall dynamics of Θ accurately.

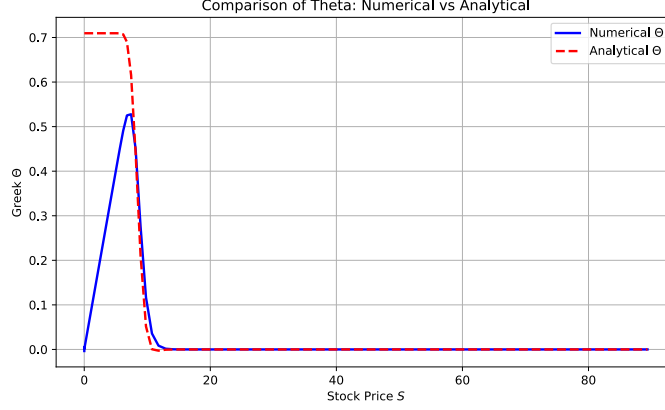


Figure 4: Comparison of the analytical Θ and the numerically approximated Θ for a vanilla European put option

In Figure 5, we return to the case of the bounded power put option with upper bound $L = \text{€}81$ and power $p = 2$, plotting $\Theta(S_i)$ for each grid point along the stock price axis to visualise the relationship. We observe that the estimated Θ reaches its maximum at approximately $S = \text{€}4.28$, which lies in the in-the-money region of the bounded power put option (with strike $K = \text{€}10$). This peak reflects the point where the option's value is growing most rapidly with time as the payoff is highly sensitive to further reductions in stock price, and the time discounting effect is strongest.

At very low stock prices ($S \rightarrow 0$), Θ declines and tends toward zero. In this region, the option is deep in-the-money and close to its capped payoff value. There is little additional value to be gained by waiting, so the change in value over time becomes small, resulting in a near-zero Θ .

As the stock price increases beyond the strike price, the option becomes out-of-the-money. Since the payoff is zero in this region and remains zero as S approaches the upper bound $L = \text{€}81$, Θ also remains close to zero, reflecting that the option is unlikely to gain value regardless of remaining time.

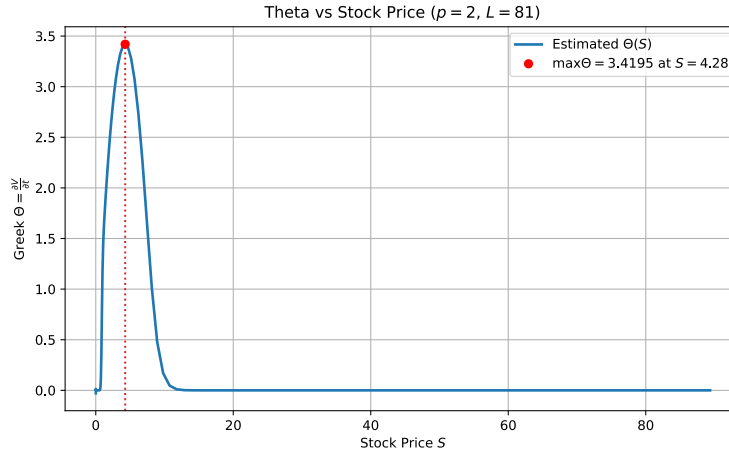


Figure 5: Estimated Greek Θ as a function of stock price S

Analysis of the Case $p = 0.5$

This section adopts the same structure and methodology as the previous one, but now focuses on a bounded power put option with an upper bound of $L = \text{€}3$ and a power of $p = 0.5$. We begin by plotting a surface plot of the option value $V(S, t)$ as a function of stock price S and time t , computed using **Algorithm 2**, as shown in Figure 6.

The power parameter $p = 0.5$ results in a concave payoff structure, capped at $L = \text{€}3$. Compared to the quadratic case, the option value increases more gradually as the stock price falls below the strike $K = \text{€}10$. The highest option values occur when the stock price is very low, but the growth in value is limited by the square-root-like shape of the concave payoff. As the stock price increases, the option value declines steadily and becomes negligible even before reaching the strike. Throughout the majority of the stock price domain, especially for prices near or above the strike, the option remains effectively worthless at all times, reflecting the low likelihood of a meaningful payoff and the capped payout structure. Unlike the $p = 2$ case, the value surface remains mostly flat and close to zero across time, with only

a modest rise in the deep in-the-money region as maturity approaches. This behaviour is consistent with the bounded concave payoff, which results in both a slower increase in value as $S \rightarrow 0$ and a lower maximum payoff at $L = \text{€}3$, leading to a generally lower option valuation across the domain.

Surface Plot of Bounded Power Put Option when $p = 0.5$ and $L = 3$

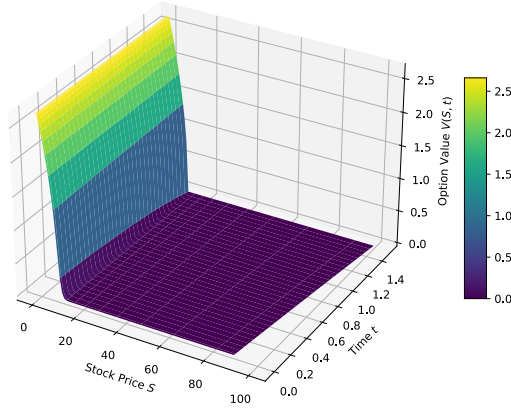


Figure 6: Surface plot of the bounded power put option value computed using the implicit method applied to the heat equation

We next compare the current value of the option, i.e. at $t = 0$, to the discounted payoff function, as shown in Figure 7. The numerical solution and the discounted payoff are in close agreement across the full range of stock prices. This confirms that the implementation accurately captures the shape and scale of the bounded power put option for the case $p = 0.5$. As expected, since the option is European-style, the value at $t = 0$ must satisfy the initial condition in Equation (19).

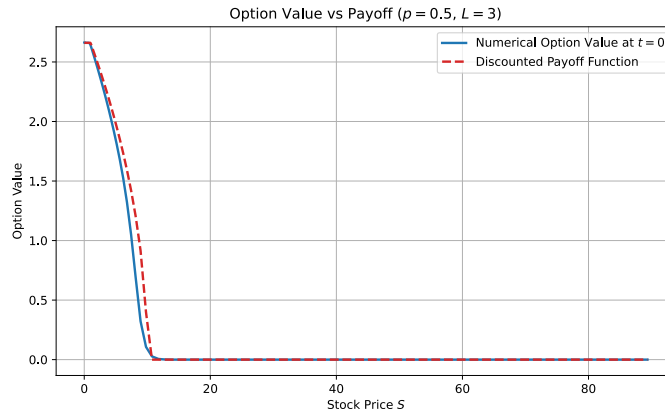


Figure 7: Comparison of the bounded power put option value and its payoff at $t = 0$

We go on to estimate the value of the option given a specific stock price of $S = \text{€}4.73$, using the same approach as in the previous section. Table 2 shows the results of our implementation. The option is in-the-money, with S moderately below the strike price $K = \text{€}10$, so a non-zero option value is expected. However, unlike the quadratic case, the power parameter $p = 0.5$ results in slower payoff growth as $K - S$ increases, due to the concave nature of the square-root payoff. The upper bound $L = \text{€}3$ has not yet been reached in this region, so the option behaves locally like an unconstrained power put. This supports both the magnitude of the estimated value and the reliability of the interpolation method used in our implementation.

S at $t = 0$ (€)	$V(S, 0)$ (€)
4.73	1.9107

Table 2: Estimated value of the bounded power put option with current stock price = $\text{€}4.73$

We now estimate the Greek Θ for our bounded power put option using the same approach as in the previous section.

In Figure 8, we plot $\Theta(S_i)$ for each grid point along the stock price axis to analyse how the option's time sensitivity varies across different values of S . We observe that the estimated Θ reaches its maximum at approximately $S = \text{€}8.15$, which lies in the in-the-money region for the bounded power put option with strike $K = \text{€}10$. Compared to

the case in Figure 5, where the maximum occurs at $S = \text{€}4.28$ for $p = 2$, the location of the peak shifts closer to the strike. This behaviour reflects the shape of the square-root-like payoff function for $p = 0.5$, which grows more slowly than the quadratic case. As a result, the option's time sensitivity is most pronounced in a narrower region just below the strike, where the payoff begins to accumulate value more gradually.

At very low stock prices ($S \ll K$), Θ decreases and tends toward zero. Here, the option is deep in-the-money and close to its capped payoff value, so further reductions in S add little value, and the rate of time decay becomes negligible. At high stock prices ($S > K$), where the option is out-of-the-money, Θ also remains close to zero, as the payoff is zero and unlikely to change. Overall, the Θ profile for $p = 0.5$ is lower in magnitude and peaks closer to the strike than in the $p = 2$ case, reflecting the square-root-like concave payoff and the lower payoff cap of $L = \text{€}3$.

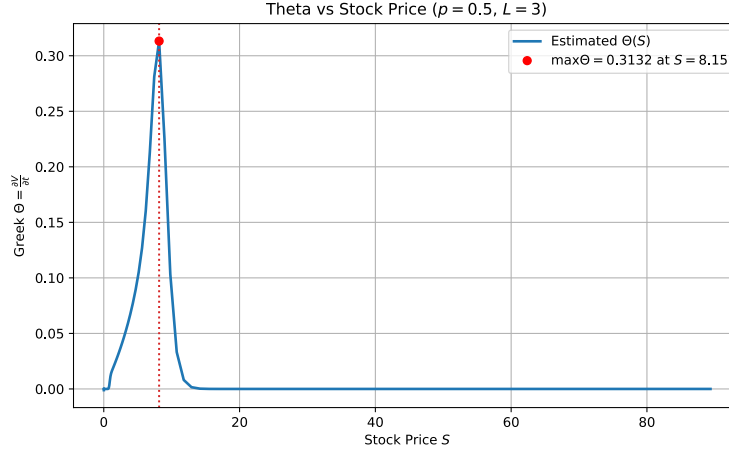


Figure 8: Estimated Greek Θ as a function of stock price S

Results and Discussion

The first two sections of our report are purely used as setup for the implementation of the implicit method for solving the Black-Scholes PDE. We are able to successfully implement a function to solve a tridiagonal system of linear equations, as well as successfully derive the boundary conditions for the bounded power put option.

Following the setup phase, we construct a function to implement the implicit finite difference scheme for solving the transformed heat equation. The solution is then mapped back to the original variables via the inverse transformation, yielding option prices on a discretised grid over stock price and time. To verify the accuracy of our implementation, we replicate the payoff structure of a vanilla European put option by setting $L = \text{€}11$ and $p = 1$, and compare our numerical results to the analytical Black-Scholes formula. Plotting the option value at the current time as a function of the stock price from $\text{€}0$ to $\text{€}20$, we observe very close agreement between the numerical and analytical solutions, confirming the correctness of our approach.

Examining the surface plots for the two cases, $L = \text{€}81$, $p = 2$ and $L = \text{€}3$, $p = 0.5$, we observe a broadly similar overall shape. In both cases, the option value remains close to zero across most of the stock price and time domain, and only rises meaningfully when the stock price is sufficiently below the strike. However, in the case with $p = 0.5$, the increase in value as the stock price decreases is more gradual, reflecting the square-root nature of the payoff function. In contrast, for $p = 2$, the quadratic payoff leads to a steeper rise in option value as the stock price falls below the strike. In both cases, the option value tends towards zero as the stock price increases beyond the strike price. Additionally, we observe that the $p = 0.5$ case results in a consistently lower valuation across the stock price and time domain, due to the slower growth and lower cap ($L = \text{€}3$) of the payoff function.

When comparing the current value of the option to the discounted payoff function, we observe a close fit in both cases, indicating correct implementation of the implicit method for solving the heat equation.

When estimating the value of the option for a current stock price of $\text{€}4.73$, the observations from the surface plots are further supported. In the case where $L = \text{€}81$ and $p = 2$, we observe a significantly higher option value compared to the case with $L = \text{€}3$ and $p = 0.5$. This difference is expected, as the quadratic payoff ($p = 2$) grows more rapidly as the stock price falls below the strike, whereas the square-root payoff ($p = 0.5$) grows more gradually due to its concave nature.

Finally, our analysis of the Greek Θ across both cases reveals similar overall shapes, but with distinct differences in the position and magnitude of the peak. For $p = 2$, the maximum Θ occurs well below the strike price, while for $p = 0.5$ it occurs closer to the strike, still within the in-the-money region. This shift reflects the differing shapes of the payoff functions: the steeper, quadratic payoff ($p = 2$) leads to sharper time decay deeper in-the-money, whereas the more gradual, square-root-like growth ($p = 0.5$) shifts the peak of time sensitivity closer to the strike. In both

cases, Θ is highest in the in-the-money region, and tends toward zero as the option becomes out-of-the-money or deep in-the-money, consistent with expected time decay behaviour.

Overall, the comparison highlights how the degree of payoff convexity, controlled by the parameter p , directly influences both the magnitude and position of the peak in Θ . A higher p amplifies the curvature of the in-the-money payoff and shifts the peak further below the strike, while a lower p reduces both the steepness and the magnitude of the peak, bringing it closer to the strike.

While the current approach is accurate and stable, it has some limitations. The Greek Θ is estimated using a forward difference in time, which is only first-order accurate and not fully aligned with the backward time-stepping nature of the implicit method. A backward difference would be more appropriate for consistency. Additionally, the implicit method, though unconditionally stable, is only first-order accurate in time. Replacing it with the Crank–Nicolson scheme, which is second-order accurate in both time and space under central differencing, could reduce numerical diffusion and improve accuracy, particularly near the strike.